

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ВГРАДЕНО ХРАНИЛИЩЕ ЗА RDF ТРИПЛЕТИ
С МЕХАНИЗЪМ ЗА ИЗПЪЛНЕНИЕ НА
ЗАЯВКИ НА SPARQL 1.1**

КУРСОВ ПРОЕКТ

по дисциплина „Семантичен уеб“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р инж. Аделина Алексиева-Петрова

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. ПРЕДМЕТНА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Информационен модел на семантичния уеб	3
1.2. Синтаксиси за обмен на RDF	3
1.3. Семантика, онтологии и правила	3
1.4. Език за заявки и подходи за индексване	4
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
2.1. Целеви език и среда за изграждане	5
2.2. Кодироване на термините	5
2.3. Набор от индекси	5
2.4. Носител на индексите	6
2.5. Вид на синтактичните анализатори	6
2.6. Стратегия на планировчика	7
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	8
3.1. Слоеви и отговорности	8
3.2. Речник на термините	8
3.3. Кодирани триплети и пермутирани индекси	9
3.4. Покритие на шаблоните от трите индекса	9
3.5. Алгебрично дърво и план за изпълнение	10
3.6. Оценка на мощността и ред на съединенията	11
3.7. Оператори за изпълнение	11
3.8. Слой за извеждане по RDFS	13
3.9. Поведение при грешка	13
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	14
4.1. Обща структура на изходния текст	14
4.2. Йерархия на типовете	15
4.3. Общ четец и синтактични анализатори	15
4.4. Анализатор на SPARQL	16

4.5. Планировчик	16
4.6. Механизъм за изпълнение	17
4.7. Извеждане по RDFS	17
4.8. Тестова рамка и генератор на данни	17
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	19
5.1. Какво се проверява	19
5.2. Коректност: синтаксис	19
5.3. Коректност: семантика на заявките	19
5.4. Набор от данни	20
5.5. Измервани величини	20
5.6. Условия за валидност на измерването	21
5.7. Конфигурация на машината	22
5.8. Какво не е измерено и защо	22
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	23
6.1. Коректност спрямо съставения набор от синтактични случаи	23
6.2. Коректност спрямо ръчно проверени отговори	23
6.3. Еднократна цена: зареждане и изграждане на индексите	24
6.4. Повтаряща се цена: изпълнение на заявки	25
6.5. Стратегия на съединяване	26
6.6. Селективност на водещия шаблон	26
6.7. Цена на извеждането по RDFS	27
6.8. Анализ	27
ЗАКЛЮЧЕНИЕ	29
ЛИТЕРАТУРА	31
ПРИЛОЖЕНИЯ	32

УВОД

Семантичният уеб описва информацията във вид, който машина може да свърже, вместо само да покаже. Основната единица на този запис е *триплетът*: подлог, сказуемо и допълнение, всяко от които е ресурс с идентификатор или литерална стойност. Моделът RDF задава абстрактния синтаксис на този запис [1], а езикът SPARQL задава как над множество триплети се формулира заявка [2]. Двете спецификации са нормативни и публични, но между тях и работеща програма стои инженерна задача, която спецификациите съзнателно не решават: как триплетите се съхраняват, индексират и обхождат така, че заявката да завърши за приемливо време.

Съществуващите реализации решават тази задача, но повечето от тях са съвърши. Те предполагат отделен процес, мрежов протокол и администриране. Има случаи, в които това е излишно: настолно приложение, вграден инструмент за анализ, конвейер за обработка на данни, който трябва да зададе няколко заявки над локален набор от триплети и да продължи. За такъв случай е подходящо *вградено* хранилище, тоест библиотека, която се свързва към приложението и работи в неговия процес и адресно пространство.

Целта на курсовия проект е да се проектира и реализира вградено хранилище за RDF триплети с механизъм за изпълнение на заявки на SPARQL 1.1, и да се измери поведението му спрямо утвърдени реализации върху едни и същи данни и заявки.

За постигането на целта са формулирани следните **задачи**:

1. преглед на информационния модел на семантичния уеб, на синтаксисите за обмен на RDF и на семантиката на SPARQL, заедно с публикуваните подходи за индексване на триплети;
2. анализ на възможните проектни решения по всяко от отворените места, тоест начин на кодиране на термините, набор от индекси, вид на синтактичния анализатор и стратегия на планировчика, с обосновка на направения избор;
3. проектиране на слоеста архитектура, в която синтаксисът, съхранението, анализът на заявки, планирането и изпълнението са отделени един от друг;
4. програмна реализация на архитектурата на C++20 с изграждане чрез CMake;
5. изграждане на методика за проверка на коректността спрямо публичните тестови набори на W3C и за измерване на времето за изпълнение спрямо еталонни реализации;

6. провеждане на измерванията, анализ на получените резултати и извеждане на заключения за приложимостта на избраните решения.

Обхватът на проекта покрива четене на Turtle и на N-Triples и записване на N-Triples, съхранение с речниково кодиране и пермутирани индекси, синтактичен анализ на подмножество на SPARQL 1.1 до алгебрично дърво, планиране на реда на съединенията и изпълнение над индексите. Правилата за извеждане по RDFS се разглеждат като отделен, незадължителен слой, реализиран в режим на материализация. Извън обхвата остават протоколът SPARQL през HTTP, езикът SPARQL Update, именуванияте графи, разпределеното изпълнение, запазването на хранилището между стартирания и пълното разсъждаване по OWL 2; последното се разглежда само описателно, доколкото дисциплината го включва. Точната граница на поддържаното подмножество на SPARQL е изброена в приложение В.

Предходна работа на автора. Дипломната работа на автора за ОКС „Бакалавър“ е библиотека на C++ за обектно релационно съответствие, изградена върху изчисления по време на компилация. Общото с настоящия проект е проектирането на език за заявки и превръщането му в изпълним план, което тук се пренася върху друг модел на данните. Изходният текст на предходната работа не се използва. **[TODO: точно заглавие и година на дипломната работа за ОКС „Бакалавър“, за цитиране]**

Структура на изложението. Раздел I разглежда предметната област и литературата. Раздел II излага разгледаните алтернативи и обосновава избора. Раздел III представя проектирането, а Раздел IV програмната реализация. Раздел V описва методиката за проверка и измерване, Раздел VI получените резултати.

I. ПРЕДМЕТНА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

1.1. Информационен модел на семантичния уеб

Класическият уеб свързва документи. Връзката между тях е хипервръзка без тип: тя казва, че два документа са свързани, но не и как. Семантичният уеб добавя точно този липсващ пласт, като описва не документите, а ресурсите, и дава на всяка връзка собствен идентификатор и значение. Единицата на записа е триплет от подлог, сказуемо и допълнение. Множество триплети образува насочен етикетирания граф, в който допълнението на един триплет може да бъде подлог на друг, и така отделно съставени описания се свързват без предварително договаряне на обща схема.

Абстрактният синтаксис на този модел е дефиниран в спецификацията RDF 1.1, която въвежда трите вида термини, тоест IRI, литерал и празен възел, и определя графа и множеството от графи [1]. Спецификацията умишлено разделя абстрактния модел от конкретните му записи. Това разделение е важно за настоящия проект: то позволява синтактичните анализатори да се разглеждат като сменяеми входове към един и същ вътрешен модел.

1.2. Синтаксиси за обмен на RDF

Обменът на RDF между системи става чрез конкретен синтаксис. Turtle е компактен запис, четим от човек, с префикси, съкратени форми за списъци и вложени описания на празни възли [3]. N-Triples е подмножество на Turtle, в което всеки ред съдържа точно един пълен триплет без съкращения [4]. Заради тази простота N-Triples е удобен за поточна обработка и за тестови набори, докато Turtle е предпочитан за ръчно писани описания. Проектът поддържа и двата, като N-Triples се третира като частен случай в същия синтактичен анализатор.

1.3. Семантика, онтологии и правила

Синтаксисът определя кои записи са допустими, но не и какво следва от тях. Семантиката на RDF задава кога един граф следва от друг, тоест отношението на извеждане [5]. RDF Schema надгражда с речник за класове, свойства, области и обхвати, и с това позволява прости правила от вида „ако ресурс принадлежи на подклас, той

принадлежи и на надкласа“ [6]. Езикът OWL 2 разширява изразителната сила значително, като добавя ограничения върху свойствата, кардиналности, дизюнктни класове и други конструкции, и с това променя изчислителната сложност на извеждането [7]. Настоящият проект реализира извеждане по RDFS като отделен слой и разглежда OWL 2 само описателно.

1.4. Език за заявки и подходи за индексирание

SPARQL 1.1 е нормативният език за заявки над RDF. Той дефинира базови графови шаблони, незадължителни части, обединения, филтри, агрегация, подзаявки и транслативни пътища по свойства, а също и алгебра, към която текстът на заявката се превежда преди изпълнение [2]. Тази алгебра е удобната граница между синтактичния анализатор и механизма за изпълнение и е приета за такава и в този проект.

Литературата за съхранение на триплети се съсредоточава върху един въпрос: базовият графов шаблон изисква многократно търсене по различни комбинации от свързани и свободни позиции, а един ред на сортиране обслужва добре само част от тях. Hexastore предлага пълния набор от шест пермутации на подлог, сказуемо и допълнение, с което всяка комбинация се обслужва от подходящо подреден индекс, срещу цената на шесткратно повторение на данните [8]. RDF-3X показва, че при речниково кодиране на термините и компресирани индекси този подход е практически приложим, и добавя планировчик, който избира реда на съединенията по статистики върху самите индекси [9]. Двете публикации формират основата на проектното решение в Раздел III.

[TODO: преглед на поне два по-нови източника за изпълнение на SPARQL, за да не е прегледът съсредоточен само в периода 2008 до 2010]

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Целеви език и среда за изграждане

Разгледани са три възможности: C++20, Rust и Java. Java отпада заради изискването хранилището да бъде вградено в приложение на друг език без отделна среда за изпълнение. Rust покрива изискването и предлага по-строги гаранции за паметта, но свързването му към съществуващ код на C++ минава през допълнителен слой. Избран е C++20, защото целевото приложение на такава библиотека най-често е на C++, защото езикът дава пряк контрол върху разположението на данните в паметта, което е определящо за индексите, и защото авторът има опит в него. Изграждането е с CMake, тъй като това е фактическият стандарт за преносими проекти на C++.

2.2. Кодиране на термините

Триплетът може да съхранява самите низове на термините или числови идентификатори, получени от речник. Прякото съхранение на низове е по-просто, но повтаря един и същ IRI толкова пъти, колкото триплета го използват, а сравненията при съединяване стават низови. Речниковото кодиране изисква два допълнителни справочника, от термин към идентификатор и обратно, но свежда триплета до три числа с фиксиран размер. Това прави индексите плътни, сравненията евтини, а съединенията сводими до сливане на сортирани редици. Избрано е речниково кодиране, следвайки [9].

2.3. Набор от индекси

Пълният набор от шест пермутации обслужва всяка комбинация от свързани позиции с подходящо подреден индекс [8], но умножава обема шест пъти. Един единствен индекс по подлог е компактен, но при шаблон със свободен подлог води до пълно обхождане. Избрани са три пермутации, а именно подлог казуемо допълнение, казуемо допълнение подлог и допълнение подлог казуемо. Всяка от осемте комбинации от свързани и свободни позиции се обслужва от префикс на поне един от трите индекса, което дава покритие без шесткратното повторение. Обосновката е обобщена в (Табл. 1).

2.4. Носител на индексите

Възможностите са структури изцяло в паметта, вграден склад от вида ключ и стойност като дърво с логаритмично сливане, и собствени файлове, отразени в паметта. Вграден външен склад отпада първи: той внася зависимост, а моделът му на съхранение не съвпада с подредените редици, които съединяването изисква.

Остават подредени редици в паметта и същите редици, отразени във файл. Двете се различават по един въпрос, а именно откъде идват байтовете, и по нищо друго: и в двата случая търсенето е двоично по префикс, а съединяването е сливане на сортирани редици. За обхвата на проекта е избран първият вариант. Отразяването във файлове изисква платформено зависим слой за вход и изход, тоест точно вида код, който трябва да бъде скрит зад абстракция и проверен на три операционни системи, а срещу това не променя нито един от алгоритмите, които този проект изследва, нито измерваните величини по време на заявка. Ограничението, което това налага, е ясно и е записано като такова: наборът от данни трябва да се събира в оперативната памет, а хранилището се изгражда наново при всяко отваряне. Отразяването във файлове и компресията на индексите остават като посока за по-нататъшна работа.

2.5. Вид на синтактичните анализатори

Граматиките на Turtle и на SPARQL 1.1 са публикувани в спецификациите и могат да бъдат подадени на генератор на анализатори. Генераторът пести писане и намалява риска от разминаване с граматиката, но добавя зависимост, стъпка в изграждането и генериран изходен текст, чиито съобщения за грешка са трудни за привеждане към полезен вид. Двете граматики са проектирани така, че да се разпознават с ограничен предварителен поглед, което ги прави удобни за анализатор с рекурсивно спускане, писан на ръка. Избран е вторият вариант, като рискът от разминаване с граматиката се покрива от набор от синтактични случаи, съставен по образа на нормативния набор на W3C [10] и описан в Раздел efsec:method.

Таблица 1. Обобщение на разгледаните алтернативи и на направения избор

Решение	Разгледани варианти	Избор	Водещ довод
Език	C++20, Rust, Java	C++20	вграждане без отделна среда за изпълнение
Термини	низове в триплета, речниково кодиране	речниково кодиране	фиксиран размер, евтини сравнения
Индекси	един, три, шест пермутации	три пермутации	покрытие на всички шаблони без шесткратен обем
Носител	памет, външен склад, отразени файлове	подредени редици в паметта	същите алгоритми, без платформено зависим вход и изход
Анализатори	генератор, рекурсивно спускане	рекурсивно спускане	по-добри съобщения, без стъпка в изграждането

2.6. Стратегия на планировчика

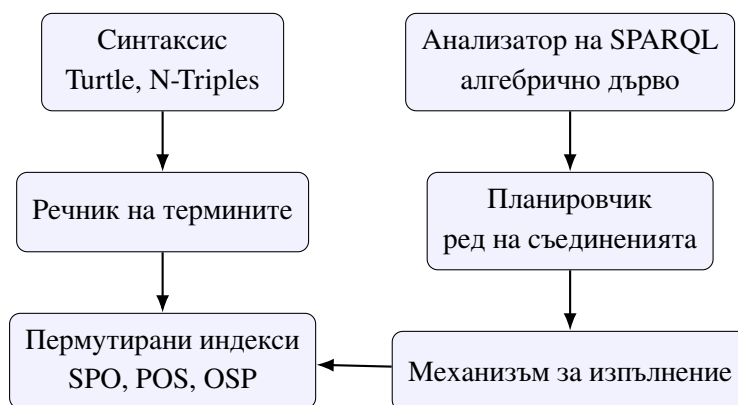
Редът на съединенията при базов графов шаблон определя размера на междинните резултати и с това времето за изпълнение. Разгледани са два подхода: фиксирано правило, което подрежда шаблоните по брой свързани позиции, и избор по оценка, основан на статистики от самите индекси. Първият е тривиален за реализация и няма цена по време на планиране, но е чувствителен към реда на изписване на заявката. Вторият изисква статистики, но остава устойчив. Избран е вторият.

Въпросът дали статистиките се поддържат постъпково при вмъкване, или се преизчисляват при отваряне на хранилището, се оказа излишен. При подредени пермутирани индекси броят на триплетите, напасващи един шаблон, е дължината на диапазона, който двоичното търсене по префикс връща, тоест той се пресмята точно за $O(\log n)$ в момента на планиране. Отделна структура със статистики не се поддържа, защото индексът вече е такава структура, а и не съществува начин оценката да се разминае с действителността.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

3.1. Слоевете и отговорности

Системата е разделена на слоеве, като всеки от тях зависи само от слоевете под себе си и общува с тях през тесен договор. Слойът на синтаксиса чете Turtle и N-Triples и превръща текста в поток от триплети над термини. Речникът на термините съпоставя всеки термин с числов идентификатор и обратно. Слойът на индексите поддържа трите пермутирани подредби на кодираните триплети. Анализаторът на SPARQL превръща текста на заявката в алгебрично дърво. Планировчикът преобразува това дърво в дърво от оператори, като избира реда на съединенията и индекса за всеки шаблон. Механизмът за изпълнение обхожда операторите и произвежда решенията. Разположението на слоевете и посоката на данните са показани на (Фиг. 1).



Фигура 1. Слоевете на системата. Лявата колона е пътят на данните при зареждане, дясната е пътят на заявката. Механизмът за изпълнение достига данните само през индексите, а планировчикът чете от тях единствено статистики

3.2. Речник на термините

Речникът поддържа две посоки. Посоката от термин към идентификатор се използва при зареждане и при превръщане на константите в заявката, а обратната само при извеждане на резултата. Ключът на съпоставянето е записът на термина в N-Triples, тъй като този запис е каноничен: един и същ термин, написан в Turtle с префикс или изцяло, дава един и същ ключ и се вписва в речника само веднъж.

Идентификаторът е 64 бита, разделени на две части. Старшите 8 бита носят вида на термина, тоест IRI, празен възел или литерал. Младшите 56 бита носят поредния номер, раздаван от речника. Това разделение позволява на механизма за изпълнение да отговори

на `isIRI`, `isLiteral` и `isBlank` без да чете самия термин, и оставя място за над $7 \cdot 10^{16}$ различни термина, което е далеч над обхвата на вграден случай на употреба. Стойността нула е запазена и означава „няма свързване“, затова поредните номера започват от единица.

Празните възли са локални за документа, от който идват. При зареждане етикетът им се предхожда от име на обхват, извлечено от името на файла, така че два документа, заредени в едно хранилище, не могат да се сблъскат. Анонимните празни възли, тоест написаните като `[]` или получени от съкратен списък, получават генерирано име в същия обхват.

3.3. Кодирани триплети и пермутирани индекси

След кодиране триpletът е три числа с фиксиран размер, тоест 24 байта. Всеки индекс е редица от такива триплети, подредена лексикографски по съответната пермутация. Поддържат се три пермутации: подлог, сказуемо, допълнение (SPO); сказуемо, допълнение, подлог (POS); и допълнение, подлог, сказуемо (OSP).

Редиците се държат изцяло в оперативната памет. Отразяването им във файлове и компресията на индексите остават извън обхвата по причината, дадена в Раздел II, и са записани като посока за по-нататъшна работа.

Търсенето по шаблон се свежда до двоично търсене на границите на префикса, съответстващ на свързаните позиции, последвано от последователно четене на диапазона. Тази форма е избрана заради две свойства: последователното четене е предвидимо за подсистемата на паметта, а подредеността позволява съединяването на два входа да бъде сливане на сортирани редици, без изграждане на хеш таблица.

3.4. Покритие на шаблоните от трите индекса

Твърдението, върху което стои изборът на точно три пермутации, е следното: за всяка от осемте комбинации от свързани и свободни позиции съществува индекс, в който всички свързани позиции образуват префикс. (Табл. 2) изброява осемте случая заедно с избрания индекс и дължината на префикса. Проверката на това твърдение е записана като тест, а не оставена като разсъждение.

Таблица 2. Избор на индекс според свързаните позиции на шаблона

Свързани позиции	Индекс	Префикс	Подредба на изхода
няма	SPO	0	по подлог
подлог	SPO	1	по сказуемо
сказуемо	POS	1	по допълнение
допълнение	OSP	1	по подлог
подлог, сказуемо	SPO	2	по допълнение
сказуемо, допълнение	POS	2	по подлог
подлог, допълнение	OSP	2	по сказуемо
и трите	SPO	3	единичен резултат

Последната колона е също толкова съществена, колкото и втората. Обхождането на диапазон в даден индекс произвежда триплетите подредени по следващата позиция от пермутацията, тоест по позицията непосредствено след префикса. Тази подредба е безплатна и е единственото условие, при което съединяването чрез сливане е допустимо.

3.5. Алгебрично дърво и план за изпълнение

Анализаторът на SPARQL произвежда дърво, чиито възли съответстват на операторите от алгебрата на спецификацията: базов графов шаблон, съединение, незадължително съединение, обединение, филтър, групиране с агрегация, проекция, премахване на повторенията, подреждане и ограничение на броя решения [2]. Планировчикът работи само върху това дърво и не познава синтаксиса, което позволява двете части да се тестват поотделно.

Планът за изпълнение е второ дърво, този път от оператори. Преобразуването прави три неща. Първо, изброява променливите на цялата заявка и дава на всяка номер на позиция, така че по време на изпълнение свързването на променлива е достъп по индекс в масив, а не търсене по име. Второ, кодира константите на всеки шаблон през речника; константа, която речникът не познава, прави шаблона празен и това се вижда преди каквото и да е обхождане. Трето, фиксира реда на съединенията и избира индекс и стратегия за всяко съединение.

3.6. Оценка на мощността и ред на съединенията

Под *мощност* на шаблон се разбира броят на триплетите, които го напасват при свързани само неговите константи. В избраната схема на индексите тази величина не се оценява приблизително, а се пресмята точно: тя е дължината на диапазона, който двоичното търсене връща, и струва $O(\log n)$. Планировчикът работи от точни числа, което премахва цял клас грешки, произтичащи от разминаване между оценка и действителност.

Редът на съединенията се избира лакомо. Първи се взема шаблонът с най-малка мощност. На всяка следваща стъпка се взема шаблонът с най-малка мощност измежду онези, които споделят променлива с вече избраните; ако такъв няма, се взема най-малкият измежду останалите. Предпочитанието към свързаност е важно, защото шаблон без обща променлива дава декартово произведение, чийто размер расте като произведение на мощностите.

За да може ползата от планирането да бъде измерена, а не предположена, планировчикът приема флаг, който запазва реда на изписване на заявката. Двата режима произвеждат едни и същи решения и се различават само по време на изпълнение.

3.7. Оператори за изпълнение

Изпълнението е верига от итератори. Всеки оператор поддържа две операции: отваряне, което получава вече направените свързвания от обхващащия оператор, и извличане на следващото решение. Този договор е причината съединяването чрез вложени цикли да бъде съединяване по индекс: вътрешният оператор получава свързванията на външния и стеснява диапазона си с тях, вместо да чете всичко и да отсява.

Таблица 3. Оператори на механизма за изпълнение

Оператор	Материализира	Забележка
Обхождане по индекс	не	избира индекс по свързаните позиции
Съединяване чрез вложени цикли по индекс	не	отваря дясната страна за всеки ред отляво
Съединяване чрез сливане	частично	буферира само групата с равен ключ
Незадължително съединяване	не	при липса на съвпадение връща реда отляво
Обединение	не	изчерпва левия вход, после десния
Филтър	не	изчислява израза върху всеки ред
Проекция	не	изчиства позициите извън проекцията
Премахване на повторения	да, ключовете	пази множество от вече върнати редове
Подреждане	да	сортиращите ключове се пресмятат веднъж на ред
Групиране с агрегация	да	никоя група не е завършена преди края на входа
Ограничение и отместване	не	спира входа рано

Материализиране се извършва само там, където то е присъщо на оператора. Подреждането не може да върне първия ред, преди да е видяло последния, а агрегацията не може да затвори група по същата причина. Останалите оператори работят поточно, което е и причината ограничението на броя решения да спира работата рано.

Съединяването чрез сливане е допустимо само когато и двата входа са обхождания по индекс, подредени по общата променлива. Според (Табл. 2) това се случва при шаблони със свързано сказуемо, които споделят допълнението си: и двата се четат от POS и излизат подредени по допълнение. В план с лява дълбочина условието може да бъде изпълнено само за първата двойка, тъй като изходът на съединение вече не е подреден.

3.8. Слой за извеждане по RDFS

Извеждането по RDFS е реализирано като материализация: правилата се прилагат след зареждане и получените триплети се вмъкват в индексите. Реализираното подмножество е следното, с номерата на правилата от спецификацията [6]:

- **rdfs2**: ако свойство има област C и съществува триплет с това свойство, подлогът му е от тип C ;
- **rdfs3**: ако свойство има обхват C , допълнението е от тип C ;
- **rdfs5**: подсвойството е транзитивно;
- **rdfs7**: триплет по подсвойство важи и по надсвойството;
- **rdfs9**: ресурс от подклас е и от надкласа;
- **rdfs11**: подкласът е транзитивен.

Правилата се прилагат до неподвижна точка. Итерирането е необходимо, защото правилата се подхранват едно друго: обхватът дава тип, а подкласът след това разширява този тип. Броят на обиколките е ограничен отгоре, за да не може дефект в правилата да доведе до безкраен цикъл вместо до съобщение за грешка.

Извеждането по време на заявка не е реализирано. То, аксиоматичните триплети на RDFS и правилата за членовете на контейнери са съзнателно извън обхвата.

3.9. Поведение при грешка

Грешките се разделят на три вида: синтактична грешка във входния документ или в заявката, нарушено съответствие в самото хранилище и изчерпан ресурс. Първият вид се съобщава с позиция в текста, ред и колона. Зареждането е двустъпково: триплетите се натрупват в междинен буфер и се прехвърлят в хранилището едва след като целият документ е прочетен, така че документ с грешка по средата не оставя частично заредени данни. Вторият вид спира операцията, защото продължаването върху повреден индекс дава тихо грешен отговор, което е по-лошо от отказ. Обхождане на хранилище, чиито индекси не са изградени след последната промяна, попада именно тук и се отхвърля.

Конструкция на SPARQL извън поддържаното подмножество се отхвърля с изрично съобщение. Пренебрегната конструкция би дала заявка, която означава нещо различно от написаното, без никакъв признак за това.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

4.1. Обща структура на изходния текст

Проектът е организиран в четири дървета. В `include/trident/` стоят публичните заглавни файлове, тоест единственото, което вижда приложението, използващо библиотеката. В `src/` стои реализацията, по един файл на слой. В `tools/` стоят трите изпълними програми, а в `tests/` тестовете. Разделянето на публично от вътрешно е съзнателно: то позволява представянето на индексите да се промени, без да се променя нито един ред в кода на потребителя на библиотеката. (Табл. 4) изброява модулите и отговорностите им.

Таблица 4. Модули на реализацията

Модул	Отговорност
<code>term</code>	термин и 64 битов идентификатор; каноничен запис в N-Triples
<code>dictionary</code>	двупосочно съпоставяне между термин и идентификатор
<code>store</code>	кодирани триплети, трите пермутирани индекса, търсене по префикс, точни мощности
<code>lexer</code>	общият четец на знаци, разрешаване на относителни IRI, четене на термини
<code>turtle</code>	анализатор на Turtle и на N-Triples; зареждане от текст и от файл
<code>sparql_ast</code>	възли на алгебричното дърво и отпечатването му
<code>sparql_parser</code>	рекурсивно спускане от текста на заявката към алгебричното дърво
<code>expression</code>	изчисляване на изрази във филтри и подредбата за сортиране
<code>planner</code>	номериране на променливите, ред на съединенията, избор на стратегия
<code>exec</code>	операторите на механизма за изпълнение
<code>rdfs</code>	материализация по правилата на RDFS
<code>query</code>	лицева функция: анализ, планиране, изпълнение, времена
<code>generator</code>	детерминиран генератор на набора от данни

Изграждането е с CMake, като библиотеката е статична, тъй като целевият случай на употреба е вграждане. Проектът няма нито една задължителна външна зависимост:

изгражда се с компилатор за C++20 и CMake и с нищо друго. Това не е въпрос на вкус. Публично хранилище, чието изграждане изисква мениджър на пакети или изтегляне по време на конфигуриране, е хранилище, което страничен наблюдател не изгражда.

4.2. Йерархия на типовете

Ключовите типове следват слоевете. Типът `Term` представя IRI, литерал или празен възел. Типът `TermId` е обвивка над 64 битово цяло число с явен конструктор, за да не могат идентификатори и обикновени числа да се разменят по невнимание. Типът `Triple` е три идентификатора, а `EncodedPattern` е три идентификатора, всеки от които може да бъде „несвързан“. Класът `PermutedIndex` държи една подредба и предоставя `prefix_range`, а `TripleStore` държи речника и трите индекса.

Възлите на алгебричното дърво са обединени в един `std::variant`, а не в йерархия с виртуални методи. Причината е, че над дървото се извършват няколко различни обхождания, тоест отпечатване, разрешаване на позициите на променливите и превод към оператори, и нито едно от тях не е естествена отговорност на самия възел. Обратно, операторите на изпълнението са йерархия с виртуални методи, защото там има точно едно поведение, тоест произвеждане на следващото решение, и то се различава при всеки оператор. Двете решения изглеждат противоположни, но следват едно и също правило: динамично свързване там, където поведението е едно и се мени, статичен разбор там, където поведението са много и възлите са затворено множество.

4.3. Общ четец и синтактични анализатори

Двата анализатора споделят `CharStream`, който води ред и колона и предоставя предварителен поглед с произволно отместване, и `TermReader`, който върху него чете IRI, префиксни имена, етикети на празни възли, низове с кавички, числови и булеви литерали. Общият слой е причината съобщенията за грешка да имат еднакъв вид и в двата случая: те винаги носят ред, колона и текст.

Анализаторът на Turtle е рекурсивно спускане по граматиката от спецификацията и поддържа префиксни обявявания в двете форми, съкратените списъци по сказуемо и по допълнение, ключовата дума `a`, вложените описания на празни възли, колекциите и четирите форми на низов литерал. N-Triples не е отделен анализатор, а същият с включен флаг, който отхвърля всяка конструкция, добавена от Turtle. Това е и начинът, по който

подмножеството се проверява: тестовият набор съдържа случаи, които трябва да бъдат приети в Turtle и отхвърлени в N-Triples.

Колекциите се разгръщат по време на анализа във верига от триплети с `rdf:first` и `rdf:rest`, завършваща с `rdf:nil`. Този избор държи модела на данните чист: под речника не съществува понятие „списък“, а само триплети.

4.4. Анализатор на SPARQL

Анализаторът на SPARQL е рекурсивно спускане с обичайната верига по приоритет: дизюнкция, конюнкция, сравнение, събиране, умножение, унарен оператор, първичен израз. Проверката за ключова дума е отделена от нейното консумиране, защото дума, последвана от двоеточие, е префиксно име, а не ключова дума, и това трябва да се различи преди да е погълнато каквото и да е.

Три решения в този модул не са очевидни. Първото: празен възел в шаблон на заявка се превежда като променлива със скрито име, съдържащо интервал. Празният възел в шаблон се държи точно като променлива, която не може да бъде избрана в проекцията, а интервал не може да се появи в написана от потребителя променлива, така че скритото име не може да бъде засенчено. Второто: филтър в началото на незадължителна група се изважда и става условие на незадължителното съединение, което е точно преводът, зададен в спецификацията. Третото: подреждането се поставя под проекцията, за да може заявката да се подрежда по променлива, която проекцията изхвърля.

4.5. Планировчик

Планировчикът обхожда алгебричното дърво веднъж, за да събере имената на всички променливи, и записва номера на позицията направо във възлите на изразите. След това изчисляването на израз не търси променлива по име нито веднъж: то чете от масив по известен индекс.

Редът на съединенията се избира по описания в Раздел III лаком алгоритъм. Изборът на индекс за вътрешната страна на съединение се прави с оглед на това какво ще бъде свързано по време на изпълнение, а не какво е свързано в текста на заявката: на конструктора на обхождането се подава списък на позициите, които обхващащият план вече ще е свързал. Затова отпечатаният план показва индекса, който наистина ще бъде използван. Кандидатът за съединяване чрез сливане, обратно, се строи без тези свързвания,

тъй като сливането изисква и двете страни да се четат независимо.

4.6. Механизъм за изпълнение

Всички оператори наследяват `Operator` с два метода: `open`, който получава реда със свързванията на обхващания оператор, и `next`, който попълва следващото решение. Редът е вектор от идентификатори с ширина, равна на броя променливи в заявката. Тази единна ширина опростява всичко останало: обединението може да свързва различни променливи в двата си клона, незадължителното съединение оставя позиции несвързани, а проекцията просто изчиства позициите извън списъка.

Съединяването чрез сливане буферира само групата редове отдясно с равен ключ, а не целия десен вход. Когато ключът отляво се смени, буферът се преизползва, ако новият ключ съвпада, и се изхвърля в противен случай.

Две места в изпълнението бяха преработени, след като измерването показва, че първоначалният им вид доминира времето. Първото е подреждането: сортиращите ключове се изчисляват веднъж на ред, а не при всяко сравнение, тъй като изчисляването включва търсене в речника и копие на термин. Второто е разчитането на числова стойност от литерал: за целочислените типове то е написано на ръка, вместо да минава през `std::stoi`, която е чувствителна към локала и с порядък по-бавна, а сравнението на числа се извиква веднъж на сравнение при подреждане и веднъж на ред при филтриране.

4.7. Извеждане по RDFS

Материализацията строи малки таблици за подклас, подсвойство, област и обхват, затваря първите две транзитивно и след това обхожда всички триплети веднъж на обиколка, като предлага следствията. Множество от вече известни триплети отсява повторенията, така че всяка обиколка добавя само нови. Обиколките спират, когато нищо ново не се появи. Схемната част на един граф е малка в сравнение с данните, затова простият алгоритъм за транзитивно затваряне е достатъчен и не е заменен с по-сложен.

4.8. Тестова рамка и генератор на данни

Тестовите използват собствена рамка от около сто и петдесет реда: макрос `TEST`, който регистрира случая чрез статичен обект, и няколко проверяващи макроса. Няма

зависимост от GoogleTest или Catch2. Изпълнимият файл приема име на група и изпълнява само нея, а CMake регистрира по един запис в CTest за всяка група.

Наборът от данни се генерира от самото хранилище. Генераторът е детерминиран: използва собствен линеен конгруентен генератор и целочислена аритметика, а не разпределенията от стандартната библиотека, чието поведение е свободно да се различава между реализации. Набор от данни, който се различава между машините, не може да служи за сравнение на измервания.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

5.1. Какво се проверява

Проверката е разделена на две независими части, защото отговарят на различни въпроси. Проверката за коректност пита дали системата дава отговора, който спецификацията изисква. Измерването на производителността пита колко струва този отговор. Втората част няма смисъл преди първата да е удовлетворена: бърз, но грешен отговор не е резултат.

5.2. Коректност: синтаксис

Синтаксисът се проверява със съставен за целта набор от случаи. Всеки случай е или положителен, тоест документ, който анализаторът трябва да приеме, или отрицателен, тоест документ, който трябва да бъде отхвърлен. Наборът покрива трите граматики: Turtle, N-Triples и поддържаното подмножество на SPARQL. Отрицателните случаи се проверяват два пъти: веднъж, че изобщо са отхвърлени, и втори път, че отказът носи ред, колона и текст на съобщението. Анализатор, който казва само „синтактична грешка“, не върши работа.

Наборът е съставен в самото хранилище, а не изтеглен. Тестовите набори на W3C [10] са именно изтегляне, а изискването към този проект е да се изгражда и проверява без никакъв мрежов достъп. Затова случаите са написани по образца на такъв набор, а не са негово копие, и това е записано изрично, за да не се чете таблицата в Раздел VI като резултат от нормативния набор. Отделно е проверено, че N-Triples наистина се държи като подмножество: всяка съкратена форма на Turtle има отрицателен случай в строгия режим.

5.3. Коректност: семантика на заявките

Семантиката се проверява върху малък граф от 40 триплета, записан в хранилището, чиито отговори са преброени на ръка от текста на файла, а не отчетени от реализацията. Проверят се базови графови шаблони, съединения по обща променлива, повторение на променлива в един шаблон, филтри с числови, низови и логически операции, незадължителни съединения заедно с BOUND, обединения, премахване на повторения, подреждане, ограничение и отместване, и всичките пет агрегиращи функции със и без

групиране.

Върху генерирания набор се проверяват твърдения, които следват от договора на генератора, например че всяка статия има точно едно заглавие, една година и един основен автор, и че сборът на груповите броеве по конференция е равен на броя на статиите.

Три свойства се проверяват отделно, защото пазят точно онова, което Раздел III твърди:

- за всяка от осемте комбинации от свързани позиции избраният индекс има префикс с дължина, равна на броя на свързаните позиции;
- планираният и непланираният ред на съединенията дават едно и също множество решения;
- съединяването чрез сливане и съединяването чрез вложени цикли дават едно и също множество решения, включително еднаква кратност на повторенията.

5.4. Набор от данни

Наборът от данни се генерира от хранилището. Той описва библиографски граф: статии с заглавие, година, конференция, тема, брой цитирания, основен автор и допълнителни автори; автори с име, месторабота и показател, като част от тях имат и домашна страница; организации, конференции и теми. Схемната част добавя подкласове, подсвойство, област и обхват, за да има слой за извеждане върху какво да работи.

Наличието на домашна страница само при част от авторите е умишлено: без него незадължителното съединение няма какво да покаже. Размерът се задава с брой статии, а останалите множества се мащабират спрямо него.

5.5. Измервани величини

Измерват се следните величини:

1. време за синтактичен анализ и кодиране на документа, и произтичащата от него пропускателна способност в триплети за секунда;
2. време за изграждане на трите индекса, отделно от времето за анализ, защото това са различни разходи: първият е четене и вписване в речник, вторият е три сортирания;
3. обем на структурите в паметта и брой различни термини в речника;

4. време за изпълнение на всяка заявка от работното натоварване, при включен и при изключен планировчик;
5. време за изпълнение при съединяване чрез сливане и при съединяване чрез вложени цикли върху една и съща заявка;
6. брой изведени триплети и време на прохода по RDFS.

Обемът се докладва като сбор от капацитета на редиците с триплети, записите на речника и техните низове. Това е величина, която програмата може да преброи точно. Върховата резидентна памет на процеса не се докладва, защото не е измерена.

Работното натоварване е осем заявки с нарастваща сложност, от единичен шаблон до верига от пет шаблона, плюс филтър, незадължително съединение, групиране и подреждане. Пълният им текст е даден в приложение Д.

5.6. Условия за валидност на измерването

Едно измерване се приема за валидно само ако са изпълнени следните условия. Всяка конфигурация се изпълнява многократно, първите изпълнения се отхвърлят, за да не се отчита загряването на разпределителя на памет и на кешовете, и се докладва медианата, а не единично измерване. Всички конфигурации четат един и същ генериран набор. Броят на решенията се отпечатва до всяко време и се сравнява между конфигурациите: ред, в който броевете не съвпадат, не се тълкува като сравнение на скорост. Времената за заявка се измерват без превръщане на решенията в текст, така че измерването да бъде механизмът за изпълнение, а не форматирането.

Измерванията са проведени на работна машина с други работещи процеси, а не на изолиран стенд. Затова целият набор е повторен три пъти и трите изхода са запазени в хранилището, а разсейването между тях е коментирано в Раздел VI вместо да бъде премълчано.

5.7. Конфигурация на машината

Таблица 5. Конфигурация, при която са проведени измерванията

Компонент	Стойност
Процесор	AMD Ryzen 5 3600, 6 ядра, 12 логически процесора, 3,95 GHz
Оперативна памет	15,9 GiB
Операционна система	Windows 11 Pro N, версия 10.0.26200
Компилятор	g++ 15.2.0, MinGW-w64, x86_64-posix-seh
Режим на изграждане	CMAKE_BUILD_TYPE=Release
Средство за изграждане	CMake 4.3.2, Ninja 1.13.2
Повторения	15 измервания, първите 2 отхвърлени, докладва се медианата

5.8. Какво не е измерено и защо

Първоначалният замисъл предвиждаше сравнение с Apache Jena [11] и RDFLib [12]. То не е проведено. И двете изискват изтегляне и среда за изпълнение, каквито на машината от (Табл. 5) не са налични, а изискването към проекта е да се изгражда и проверява без мрежов достъп. Понеже число, което не е измерено, няма място в отчет, съответните клетки не са попълнени с предположения, а сравнението е обявено за непроведено. Работното натоварване и генераторът обаче са записани така, че сравнението да може да бъде проведено по-късно без промяна в кода: наборът се записва като файл на Turtle, а заявките са обикновен текст на SPARQL.

Не са измерени и следните величини: върхова резидентна памет на процеса, поведение при набор, по-голям от оперативната памет, и цена на извеждането по време на заявка, тъй като този режим не е реализиран.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Числата в този раздел са измерени на машината от (Табл. 5) с командите, дадени в приложение Е. Суровият изход на измерителната програма е запазен в docs/measurements/ в три файла, по един на изпълнение. Таблиците възпроизвеждат второто изпълнение, а разсейването между трите е коментирано изрично там, където е съществено.

6.1. Коректност спрямо съставения набор от синтактични случаи

Таблица 6. Резултати от съставения набор от синтактични случаи

Граматика	Общо	Положителни	Отрицателни	Преминали	Непреминали
Turtle	41	26	15	41	0
N-Triples	16	7	9	16	0
SPARQL 1.1	44	26	18	44	0
Общо	101	59	42	101	0

Нито един случай не е пропуснат. Всичките 42 отрицателни случая са отхвърлени с изключение, носещо ред, колона и текст, което е проверено отделно. Осемнадесетте отрицателни случая за SPARQL включват конструкциите извън обявеното подмножество, тоест ASK, CONSTRUCT, DESCRIBE, GRAPH, BIND, VALUES, HAVING и пътищата по свойства. Те не са дефекти, а обявени граници, и тестът проверява именно това: че границата се съобщава, вместо да бъде премълчана.

Таблицата се отнася за съставения в хранилището набор, не за нормативния набор на W3C [10], по причината, дадена в Раздел V.

6.2. Коректност спрямо ръчно проверени отговори

Пълният набор от тестове съдържа 125 случая в 10 групи, регистрирани поотделно в CTest. При изпълнението, документирано в приложение Е, преминават всичките 125. (Табл. 7) показва разпределението им.

Таблица 7. Групи тестове и брой случаи във всяка

Група	Случаи	Какво покрива
dictionary	10	кодиране, обратно четене, различаване на литерали по език и по тип
turtle	21	конструкциите на Turtle, строгият режим за N-Triples, позиция на грешката
store	11	подреденост на трите индекса, покритие на осемте комбинации, точни мощности
sparql_parser	19	форма на алгебричното дърво, приоритет на операциите, отхвърляне на неподдържаното
planner	11	избор на индекс, ред на съединенията, избор и изключване на сливането
exec	20	резултати от заявки върху граф с ръчно преброени отговори
aggregate	9	петте агрегиращи функции, групиране, празен вход
rdfs	9	шестте правила, транзитивност, идемпотентност, цикъл в йерархията
syntax_suite	4	съставеният набор от 101 синтактични случая
demo_queries	11	твърдения върху генерирания набор и съгласуваност между стратегиите
Общо	125	

6.3. Еднократна цена: зареждане и изграждане на индексите

Таблица 8. Зареждане, изграждане на индексите и обем на структурите

Статии	Триплети	Вход, В	Анализ, ms	Триплети/s	Индекси, ms	Обем, KiB
1 000	10 462	301 893	12,2	858 239	2,0	1 827
5 000	52 395	1 528 499	59,4	882 778	11,1	7 024
20 000	209 202	6 185 046	261,4	800 498	57,4	28 032
50 000	522 910	15 546 537	504,6	1 036 328	114,5	72 123

Броят на различните термини в речника е съответно 2 135, 7 614, 27 565 и 67 501.

Три неща личат от таблицата. Първо, времето за анализ расте линейно с броя на триплетите, а пропускателната способност остава от порядъка на 800 000 до 1 000 000 триплета за секунда, без спад при петдесеткратно нарастване на входа. Второ,

изграждането на трите индекса е около една пета от времето за анализ, тоест трите сортирания не са водещият разход при зареждане, въпреки че всяко от тях преминава през целия набор. Трето, обемът расте линейно и е около 141 байта на триплет при най-големия набор, което включва и трите копия на триплета по 24 байта, и речника с низовете в него.

Разсейването между трите изпълнения е най-голямо именно тук. Пропускателната способност при 20 000 статии е 474 237, 800 498 и 905 522 триплета за секунда в трите изпълнения, тоест разлика от почти два пъти. Причината е, че измерванията са проведени на работна машина, а не на изолиран стенд, и че всяко повторение заема и връща десетки мегабайта памет. Величината следва да се чете като порядък, а не като точна стойност.

6.4. Повтаряща се цена: изпълнение на заявки

Таблица 9. Медианно време за изпълнение при включен и при изключен планировчик, 209 202 триплета

Заявка	Планирано, ms	Наивно, ms	Отношение	Решения
B1 един шаблон	1,375	1,400	1,02	20 000
B2 звезда от два шаблона	6,822	6,831	1,00	20 000
B3 верига от четири шаблона	0,028	12,705	447,4	47
B4 верига от пет шаблона	0,028	12,833	465,0	0
B5 филтър по число	6,172	6,122	0,99	3 188
B6 незадължително съединение	1,352	1,348	1,00	4 000
B7 групиране с брояч	2,613	2,918	1,12	200
B8 подреждане на цялото	13,623	13,894	1,02	20 000

Колоната с броя решения не е допълнителна информация, а условие за валидност: двата режима връщат едно и също множество решения при всяка заявка, което е проверено и от измерителната програма, и от отделен тест. Нулата при B4 е действителният отговор на тази заявка върху този набор, а не пропуснато измерване: авторите от избраната организация нямат статия в избраната конференция. Редът остава в таблицата, защото времето, което двата режима харчат, за да стигнат до празния отговор, се различава 465 пъти.

6.5. Стратегия на съединяване

Таблица 10. Съединяване чрез сливане срещу съединяване чрез вложени цикли по индекс

Стратегия	Исп. 1, ms	Исп. 2, ms	Исп. 3, ms
Съединяване чрез сливане	10,313	8,995	9,149
Съединяване чрез вложени цикли	10,112	10,049	10,042

Заявката е с два шаблона със свързано казуемо, споделящи допълнението си, тоест единствената форма, при която сливането е допустимо. И двете стратегии връщат 99 996 решения. Тук се докладват и трите изпълнения, защото разликата между стратегиите е от порядъка на разсейването: в две от трите изпълнения сливането е по-бързо с около 10 на сто, а в едното е по-бавно с 2 на сто. Заключение, което тези числа позволяват, е че при този размер на резултата стратегията на съединяване не е водещият разход, а той е произвеждането и връщането на близо сто хиляди решения.

6.6. Селективност на водещия шаблон

Таблица 11. Влияние на мощността на водещия шаблон върху времето за изпълнение

Константа	Мощност	Време, ms	Решения
ex:org1	4	0,022	29
ex:org5	5	0,022	30
ex:org3	11	0,028	47

Времето расте с мощността на водещия шаблон, а не с размера на набора: и трите заявки се изпълняват над едни и същи 209 202 триплета и отнемат под три стотни от милисекундата. Това е пряката проверка на решението от Раздел III, че редът на съединенията се избира по точна мощност: разходът се определя от най-селективния шаблон, а останалите се обхождат по индекс.

6.7. Цена на извеждането по RDFS

Таблица 12. Материализация по RDFS върху 209 202 триплета

Величина	Стойност
Триплети преди прохода	209 202
Изведени триплети	68 398
Триплети след прохода	277 600
Обиколки до неподвижна точка	2
Време на прохода, три изпълнения, ms	168,6 / 312,3 / 157,5
Обем преди прохода, KiB	28 032
Обем след прохода, KiB	38 985

Материализацията увеличава набора с 32,7 на сто и обема с 39,1 на сто, срещу еднократен разход, съизмерим с времето за зареждане на същия набор. Двете обиколки не са случайност: първата прилага обхвата и подсвойството, втората разширява получените типове по подклас.

Сравнение с извеждане по време на заявка не се докладва, защото този режим не е реализиран.

6.8. Анализ

Планирането има смисъл там, където има какво да се планира. При заявки с един или два шаблона планираният и наивният ред съвпадат или се различават в границите на шума, което се вижда от отношенията около единица при B1, B2, B5, B6 и B8. При вериги от четири и пет шаблона отношението е между 447 и 465 пъти. Причината е пряка: при наивен ред първият шаблон е `?p ex:title ?t` с мощност 20 000, докато при планиран ред първи е `?a ex:affiliation ex:org3` с мощност 11. Междинният резултат тръгва с три порядъка по-малък и всяко следващо обхождане е търсене по префикс, а не пълен обход. Разходът за самото планиране е под една стотна от милисекундата и не се вижда в таблицата.

Точната мощност се оказва и по-проста, и по-надеждна от оценка. Първоначалният замисъл предвиждаше поддържане на статистики. Оказа се, че самата структура на индекса вече е статистиката: двоично търсене дава точния брой съвпадения

за $O(\log n)$, така че нищо не се поддържа отделно и няма как оценката да се разминае с действителността.

Три пермутации се оказаха достатъчни, но не и безплатни. Покритието на осемте комбинации е пълно, което е проверено. Цената е трикратно повторение на триплетите, тоест 72 от 141 байта на триплет. Останалото е речникът, чийто дял расте, когато низовете са дълги.

Стратегията на съединяване не се оказа водеща. Разликата между сливане и вложени цикли е в границите на разсейването при резултат от сто хиляди решения. Това е резултат, който противоречи на очакването при проектирането, и е записан такъв, какъвто е. Обяснението, което числата допускат, е че при план с лява дълбочина сливането е приложимо само за първата двойка шаблони, а разходът се измества към произвеждането на решенията.

Причината за разход трябва да се измерва, а не да се предполага. При разработката заявка с подреждане върху 1 500 реда беше значително по-бавна от очакваното. Направени бяха две промени. Сортиращите ключове бяха изнесени така, че да се изчисляват веднъж на ред, а не при всяко сравнение, тъй като изчисляването включва търсене в речника и копие на термин. Разчитането на числовата стойност на целочислен литерал беше написано на ръка, вместо да минава през `std::stod`. Върху текущия код същата заявка се изпълнява за 1,3 милисекунди. Междинните състояния на кода не са запазени в хранилището и времената им не могат да бъдат възпроизведени с команда от него, затова тук не се докладват числа за тях: **[TODO: време преди двете промени и след първата от тях, ms]**. И двете промени останаха в кода.

Ограничения на самите измервания. Те са проведени на работна машина, при която разсейването между изпълнения достига два пъти за пропускателната способност при зареждане. Затова числата за зареждане следва да се четат като порядък. Числата за заявки са значително по-устойчиви, тъй като всяко измерване е кратко и се повтаря петнадесет пъти. Сравнение с външни реализации не е проведено и това е обявено в Раздел V.

ЗАКЛЮЧЕНИЕ

Проектът разглежда една задача от предметната област на семантичния уеб, а именно съхранението на RDF триплети и изпълнението на заявки над тях, и я решава във вид, подходящ за вграждане в приложение. Избраната архитектура разделя синтаксиса, съхранението, анализа на заявката, планирането и изпълнението на слоеве с тесни граници помежду им, което позволява всеки от тях да бъде проверяван поотделно. Реализацията се изгражда с компилатор за C++20 и CMake и няма нито една задължителна външна зависимост.

Предимства на избраното решение. Речниковото кодиране и подредените пермутирани индекси свеждат съединяването до обхождане на диапазони и премахват низовите сравнения от горещия път. Трите пермутации покриват всичките осем комбинации от свързани позиции, което е проверено, а не прието. Точната мощност на шаблон се получава от самия индекс за $O(\log n)$, така че планировчикът работи от истински числа и не поддържа отделни статистики. Липсата на външни зависимости прави вграждането и възпроизвеждането на резултатите прости.

Недостатъци и ограничения. Индексите се държат изцяло в паметта и се изграждат наново при всяко отваряне, тоест наборът трябва да се събира в оперативната памет. Трикратното повторение на триплета струва 72 от измерените 141 байта на триплет. Три пермутации вместо шест означават, че част от шаблоните се обслужват от индекс, който не е подреден по най-удобния за тях начин. Поддържаното подмножество на SPARQL не покрива именуван граф, подзаявки, пътища по свойства, BIND, VALUES и HAVING. Извеждането по RDFS е само в режим на материализация.

Инженерна трактовка на измерените резултати. Таблиците от Раздел VI допускат три извода.

Първо, планирането на реда на съединенията си заслужава точно при веригите от три и повече шаблона: там отношението между непланирания и планирания ред е между 447 и 465 пъти, докато при заявки с един или два шаблона то е в границите на шума. Разходът за самото планиране е под една стотна от милисекундата и не се вижда в никое от измерванията. От това следва практическото правило, че планировчик от този вид няма смисъл да се прави изключваем в производствен код: той никога не струва повече от това, което спестява.

Второ, стратегията на съединяване се оказва второстепенна. Разликата между сливането и вложените цикли по индекс е от порядъка на разсейването между изпълненията, при резултат от близо сто хиляди решения. Това противоречи на очакването при проектирането и е записано такова, каквото е. Обяснението, което числата допускат, е че при план с лява дълбочина сливането е приложимо само за първата двойка шаблони, а водещият разход се измества към произвеждането на самите решения.

Трето, за разглеждания случай на употреба, тоест вграден инструмент над локален набор, измерените величини са в приемливите граници: около 800 000 до 1 000 000 триплета за секунда при зареждане, около 141 байта на триплет и време под три стотни от милисекундата за селективна заявка над 209 202 триплета. Заявките, чието време е от порядъка на десет милисекунди, са именно онези, които връщат двадесет хиляди решения, тоест разходът е в резултата, а не в достъпа до данните.

Ограничението на тези изводи е, че всички измервания са на една реализация и на една машина. Сравнението с утвърдени реализации не е проведено и причината за това е записана в Раздел V, а не заобиколена с предположения.

Предложения за по-нататъшна работа. Естествените продължения са пет: отразяване на индексите във файлове, за да може хранилище да се отваря без повторно изграждане; компресия на индексите, която да намали обема без да разруши възможността за двоично търсене по префикс; извеждане по RDFS по време на заявка, за да може двата режима да бъдат сравнени, както предвиждаше първоначалният замисъл; поддръжка на именувани графи заедно със съответното разширяване на индексите до четворки; и провеждане на сравнението с външни реализации, за което работното натоварване и генераторът на данни вече са готови.

ЛИТЕРАТУРА

- [1] R. Cyganiak, D. Wood, and M. Lanthaler, “RDF 1.1 concepts and abstract syntax.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/rdf11-concepts/>.
- [2] S. Harris and A. Seaborne, “SPARQL 1.1 query language.” W3C Recommendation, Mar. 2013. <https://www.w3.org/TR/sparql11-query/>.
- [3] E. Prud’hommeaux and G. Carothers, “RDF 1.1 Turtle: Terse RDF triple language.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/turtle/>.
- [4] G. Carothers and A. Seaborne, “RDF 1.1 N-Triples: A line-based syntax for an RDF graph.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/n-triples/>.
- [5] P. J. Hayes and P. F. Patel-Schneider, “RDF 1.1 semantics.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/rdf11-mt/>.
- [6] D. Brickley and R. V. Guha, “RDF Schema 1.1.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/rdf-schema/>.
- [7] W3C OWL Working Group, “OWL 2 Web Ontology Language document overview (second edition).” W3C Recommendation, Dec. 2012. <https://www.w3.org/TR/owl2-overview/>.
- [8] C. Weiss, P. Karras, and A. Bernstein, “Hexastore: sextuple indexing for semantic web data management,” *Proceedings of the VLDB Endowment*, vol. 1, no. 1, pp. 1008–1019, 2008.
- [9] T. Neumann and G. Weikum, “The RDF-3X engine for scalable management of RDF data,” *The VLDB Journal*, vol. 19, no. 1, pp. 91–113, 2010.
- [10] W3C SPARQL Working Group, “SPARQL 1.1 test suite,” 2013. <https://www.w3.org/2009/sparql/docs/tests/>.
- [11] The Apache Software Foundation, “Apache Jena documentation,” **[TODO: година на използваната версия]**. Версия: **[TODO: версия на Jena, използвана при измерванията]**. <https://jena.apache.org/documentation/>.
- [12] RDFLib Team, “RDFLib documentation,” **[TODO: година на използваната версия]**. Версия: **[TODO: версия на RDFLib, използвана при измерванията]**. <https://rdflib.readthedocs.io/>.

ПРИЛОЖЕНИЯ

Приложение А. Представяне на индексите

Всеки индекс е непрекъсната редица от записи по 24 байта, подредена лексикографски по своята пермутация. Записът е три 64 битови идентификатора в реда подлог, сказуемо, допълнение; пермутацията определя реда на сравнение, а не реда на съхранение, така че трите индекса имат едно и също оформление на записа и се различават само по подредба.

Таблица 13. Оформление на един запис в индекс

Отместване	Размер	Съдържание
0	8	идентификатор на подлога
8	8	идентификатор на сказуемото
16	8	идентификатор на допълнението

Пермутациите са SPO с ред на сравнение (0, 1, 2), POS с ред (1, 2, 0) и OSP с ред (2, 0, 1). Редиците се държат в оперативната памет и се изграждат наново при отваряне; двоичен формат на диска и признак за версия няма, тъй като запазването на хранилището между стартирания остава извън обхвата.

Приложение Б. Представяне на речника

Речникът поддържа две структури. Правата посока е хеш таблица от каноничния запис на термина в N-Triples към идентификатор. Обратната посока е вектор от записи на термини, индексирани с пореден номер минус едно.

Идентификаторът е 64 бита: старши 8 бита за вида на термина и младши 56 бита за пореден номер. Видовете са 1 за IRI, 2 за празен възел и 3 за литерал. Стойността нула е запазена за „няма свързване“ и не се раздава, затова поредните номера започват от 1.

Записът на термина съдържа вида, лексикалната форма, езиковия етикет и типа на данните. Литерал без явен тип получава `xsd:string`, а литерал с езиков етикет получава `rdf:langString`, както изисква RDF 1.1. Затова две различни изписвания на един и същ термин се вписват в речника само веднъж.

Приложение В. Поддържано подмножество на SPARQL

Разпознава се: обявявания PREFIX и BASE; SELECT с изброени променливи или *; DISTINCT и REDUCED; базови графови шаблони със съкратени списъци по сказуемо и по допълнение и с ключовата дума а; литерали с езиков етикет и с тип, числови и булеви литерали; празни възли във вид _ : етикет и [] ; вложени групи; OPTIONAL; UNION; FILTER; GROUP BY по променливи; агрегиращите функции COUNT, SUM, MIN, MAX, AVG и SAMPLE, със и без DISTINCT, включително COUNT(*); ORDER BY с ASC и DESC; LIMIT и OFFSET.

В изразите се разпознават: | |, &&, =, !=, <, >, <=, >=, +, -, *, /, унарни ! и -; и функциите BOUND, STR, LANG, DATATYPE, isIRI, isURI, isLITERAL, isBLANK, isNUMERIC, STRLEN, UCASE, LCASE, CONTAINS, STRSTARTS, STREND, SUBSTR, REGEX, ABS, CEIL, FLOOR, ROUND, IF, COALESCE, sameTerm и LANGMATCHES.

Отхвърля се с изрична грешка: ASK, CONSTRUCT, DESCRIBE, GRAPH, SERVICE, MINUS, BIND, VALUES, HAVING, подзаявки, пътища по свойства, GROUP BY по израз, както и всяко име на функция извън списъка по-горе. Всеки от тези случаи има отрицателен запис в набора от синтактични случаи, тоест границата на подмножеството е проверявана, а не само описана.

Приложение Г. Изходен текст на избрани модули

Изборът на индекс по свързаните позиции на шаблона. Функцията брои колко водещи позиции на всяка пермутация са свързани и връща пермутацията с най-дълъг такъв префикс.

```
1 IndexChoice choose_index(const EncodedPattern& pattern) {
2     const IndexOrder orders[3] = {IndexOrder::Spo, IndexOrder::Pos,
   ↪ IndexOrder::Osp};
3     IndexChoice best;
4     best.prefix_len = -1;
5     for (IndexOrder order : orders) {
6         const std::array<int, 3> perm = permutation_of(order);
7         int len = 0;
8         while (len < 3 && pattern.bound(perm[len])) ++len;
9         if (len > best.prefix_len) {
10             best.order = order;
11             best.prefix_len = len;
12         }
13     }
14     return best;
15 }
```

Листинг 1: Избор на индекс, src/store.cpp

Отварянето на обхождане по индекс. Позициите, свързани от обхващащия оператор, се вливат в ключа за търсене, което превръща вложениия цикъл в съединяване по индекс.

```
1 void IndexScan::open(const Row& input) {
2     input_ = input;
3     rows_produced_ = 0;
4     cursor_ = end_ = 0;
5     if (pattern_.impossible) return;
6
7     EncodedPattern effective;
8     TermId parts[3];
9     for (int i = 0; i < 3; ++i) {
10         parts[i] = pattern_.constant[i];
11         if (!parts[i].valid() && pattern_.slot[i] >= 0) {
12             TermId bound = input_[static_cast<std::size_t>(pattern_.slot[i])]
   ↪ ];
13             if (bound.valid()) parts[i] = bound;
14         }
15     }
16     effective.s = parts[0];
17     effective.p = parts[1];
18     effective.o = parts[2];
```

```

19
20     IndexChoice choice = choose_index(effective);
21     index_ = &store_.index(choice.order);
22     PermutedIndex::Range range =
23         index_->prefix_range(Triple{effective.s, effective.p, effective.o},
↪ choice.prefix_len);
24     cursor_ = range.begin;
25     end_ = range.end;
26 }

```

Листинг 2: Отваряне на обхождане, src/exes.cpp

Лакомият избор на ред на съединенията. Предпочита се шаблон, свързан с вече избраните, а измежду тях този с най-малка мощност.

```

1  std::vector<std::size_t> order_patterns(const std::vector<CompiledPattern>&
↪ patterns,
2
3                                     const std::vector<std::size_t>&
↪ cardinalities) {
4      std::vector<std::size_t> order;
5      if (options_.naive_join_order) {
6          for (std::size_t i = 0; i < patterns.size(); ++i) order.push_back(i);
7          return order;
8      }
9      std::vector<bool> used(patterns.size(), false);
10     std::vector<int> bound;
11     for (std::size_t step = 0; step < patterns.size(); ++step) {
12         std::size_t best = patterns.size();
13         bool best_connected = false;
14         for (std::size_t i = 0; i < patterns.size(); ++i) {
15             if (used[i]) continue;
16             bool connected = !bound.empty() && shares_variable(patterns[i],
↪ bound);
17             if (best == patterns.size()) {
18                 best = i;
19                 best_connected = connected;
20                 continue;
21             }
22             if (connected != best_connected) {
23                 if (connected) { best = i; best_connected = true; }
24                 continue;
25             }
26             if (cardinalities[i] < cardinalities[best]) best = i;
27         }
28         used[best] = true;
29         order.push_back(best);
30         pattern_variables(patterns[best], bound);
31     }
32     return order;
33 }

```

Листинг 3: Ред на съединенията, src/planner.cpp

Съединяване чрез сливане. Буферира се само групата редове отдясно с равен ключ, след което тя се използва, докато ключът отляво не се смени.

```
1 bool MergeJoin::next(Row& out) {
2     const std::size_t slot = static_cast<std::size_t>(join_slot_);
3     for (;;) {
4         if (block_active_) {
5             if (block_cursor_ < right_block_.size()) {
6                 const Row& right = right_block_[block_cursor_++];
7                 out = left_row_;
8                 for (std::size_t i = 0; i < out.size(); ++i) {
9                     if (!out[i].valid()) out[i] = right[i];
10                }
11                ++rows_produced_;
12                return true;
13            }
14            left_valid_ = advance_left();
15            block_cursor_ = 0;
16            if (!left_valid_ || right_block_.empty() ||
17                left_row_[slot] != right_block_.front()[slot]) {
18                block_active_ = false;
19                right_block_.clear();
20            }
21            continue;
22        }
23        if (!left_valid_ || !right_valid_) return false;
24        TermId lk = left_row_[slot];
25        TermId rk = right_row_[slot];
26        if (lk < rk) { left_valid_ = advance_left(); continue; }
27        if (rk < lk) { right_valid_ = advance_right(); continue; }
28        right_block_.clear();
29        while (right_valid_ && right_row_[slot] == lk) {
30            right_block_.push_back(right_row_);
31            right_valid_ = advance_right();
32        }
33        block_cursor_ = 0;
34        block_active_ = true;
35    }
36 }
```

Листинг 4: Сливане на два подредени входа, src/exec.cpp

Приложение Д. Заявки, използвани при измерванията

Всички заявки започват с PREFIX ex: <http://example.org/>, който е пропуснат по-долу заради мястото.

```
1 B1 SELECT ?p ?t WHERE { ?p ex:title ?t }
2
3 B2 SELECT ?t ?y WHERE { ?p ex:title ?t . ?p ex:year ?y }
4
5 B3 SELECT ?t ?n WHERE { ?p ex:title ?t . ?p ex:mainAuthor ?a .
6                       ?a ex:name ?n . ?a ex:affiliation ex:org3 }
7
8 B4 SELECT ?t ?n WHERE { ?p ex:title ?t . ?p ex:mainAuthor ?a .
9                       ?a ex:name ?n . ?a ex:affiliation ex:org3 .
10                      ?p ex:venue ex:venue2 }
11
12 B5 SELECT ?p WHERE { ?p ex:citationCount ?c FILTER (?c > 100) }
13
14 B6 SELECT ?n ?h WHERE { ?a a ex:Author . ?a ex:name ?n
15                      OPTIONAL { ?a ex:homepage ?h } }
16
17 B7 SELECT ?v (COUNT(?p) AS ?n) WHERE { ?p ex:venue ?v } GROUP BY ?v
18
19 B8 SELECT ?p WHERE { ?p ex:citationCount ?c } ORDER BY DESC(?c)
20
21 Заявка за стратегията на съединяване:
22 SELECT ?a ?first ?second WHERE { ?first ex:mainAuthor ?a .
23                                ?second ex:author ?a }
```

Листинг 5: Работно натоварване

Приложение Е. Указания за изграждане и стартиране

Изисква се компилатор за C++20 и CMake 3.20 или по-нов. Външни зависимости няма и по време на конфигуриране не се тегли нищо. Командите по-долу са изпълнени на машината от (Табл. 5) и са възпроизведени дословно в README.md на хранилището.

```
1 cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
2 cmake --build build
3 ctest --test-dir build --output-on-failure
4
5 build/trident_demo
6 build/trident_bench --repeats 15
7
8 cd docs && latexmk -pdf Main.tex
```

Листинг 6: Изграждане, тестове, демонстрация и измервания

Изпълнението на `ctest` докладва 10 записа, съответстващи на групите от (Табл. 7), и 125 отделни случая. Демонстрационната програма генерира набора от данни, зарежда го, изпълнява десет заявки с нарастваща сложност и за всяка отпечатва алгебричното дърво, избрания план, броя решения и изтеклото време, след което сравнява планирания с наивния ред и накрая прилага прохода по RDFS.